

Lab 5.2 Middleware 2 - MTConnect Adapter

2.1 Multiple MTConnect Adapters

In this part, we will run multiple MTConnect adapters so that each adapter is responsible for each sensor data collection. The schematic of this case is shown in Figure 10. Because we will collect data from three different sensors with different sampling periods each, three adapters, `lab5_adapter2-1.py`, `lab5_adapter2-2.py`, and `lab5_adapter2-3.py`, are needed.

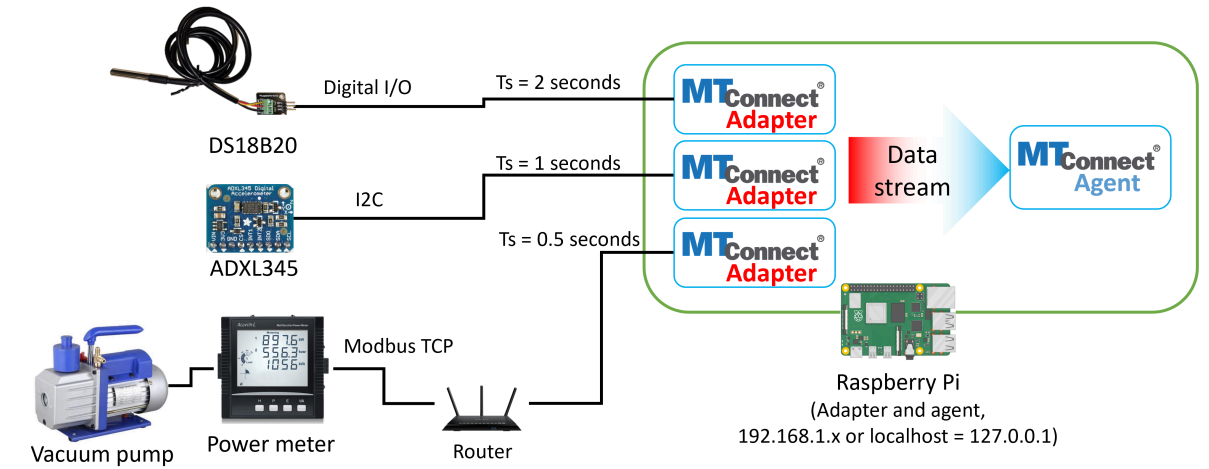


Figure 10 MTConnect configuration 2: Multiple adapter case

Please follow the steps below.

1. Halt MTConnect agent (Shortcut: Ctrl + c on the 'Terminal')
 2. Modify 'agent.cfg' file to have 3 adapters as Figure 11.
- a. Please note that each adapter **MUST** have **different port number**.

```

Devices = Device.xml
Port = 5000
AllowPut = true
ReconnectInterval = 1000
BufferSize = 17
MonitorConfigFiles = true
Pretty = false
SchemaVersion = 1.5

Adapters {
  # Log file has all machines with device name prefixed
  Adapter1 {
    # This is for DS18B20 & Virtual sensor
    Host = 127.0.0.1
    Port = 7878
  }

  Adapter2 {
    # This is for ADXL345 sensor
    Host = 127.0.0.1
    Port = 7879
  }

  Adapter3 {
    # This is for power meter
    Host = 127.0.0.1
    Port = 7880
  }
}

```

Figure 11 'agent.cfg' for multiple MTConnect adapters

3. Don't modify 'Device.xml'.
- a. We will use the same as before. 4. Run the agent again.

Now, we are ready to run the adapter.

Task 2.1

1. Modify each sample adapter program to make all complete to have all values available. And then run all the adapters.
 2. Capture a web browser as Figure 9 after requesting 'current' from the agent and attach it to the report below.
-

• creationTime: 2026-02-24T06:01:38Z
 • sender: robertclaud
 • instanceId: 1771912823
 • version: 1.8.0.3
 • bufferSize: 131072
 • nextSequence: 192
 • firstSequence: 1
 • lastSequence: 191

Device: sensor; UUID: 000

Device : sensor

Samples

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T06:01:37.409959Z	Acceleration		Xacc	a1	189	-0.2745862
2026-02-24T06:01:36.408519Z	Acceleration		Yacc	a2	186	-0.588399
2026-02-24T06:01:38.411483Z	Acceleration		Zacc	a3	191	10.002783
2026-02-24T06:01:37.007833Z	HumidityRelative	ACTUAL	humd	h1	188	53.2959602611621
2026-02-24T06:01:31.206372Z	Wattage	ACTUAL	power	p1	175	60
2026-02-24T06:01:37.007833Z	Temperature	ACTUAL	temp	t1	187	17.375

Events

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T06:00:23.568923Z	Availability			avail	14	AVAILABLE
2026-02-24T06:01:30.650093Z	PowerState	LINE	powerstate	ps	174	ON
2026-02-24T06:00:23.515548Z	AssetChanged			s1_asset_chg	2	UNAVAILABLE
2026-02-24T06:00:23.513839Z	AssetRemoved			s1_asset_rem	1	UNAVAILABLE

Task 2.2

Place the entire Python code of each adapter to the report below in each appropriate section

```
In [ ]: # Lab5_adapter2-1.py

# Adapter for DS18B20 sensor
# Sampling period = 2 sec

# Place your code here:
# MTConnect adapter sample for ME597 Lab5
# This code is for virtual sensor (random humidity value)
#           and temperature sensor (measure temperature)

import sys
import time
import datetime
from data_item import Event, Sample # Load data_item package
from mtconnect_adapter import Adapter # Load mtconnect_adapter package

import os
import glob
import random

class MTConnectAdapter(object): # MTConnect adapter object

    def __init__(self, host, port): # init of MTconnectAdapter class
        # MTConnect adapter connection info
        self.host = host # host arg of adapter
        self.port = port # port arg of adapter
        self.adapter = Adapter((host, port))

        # For samples
        # DS18B20 temperature sensor
        self.t1 = Sample("t1") # self.t1 takes 't1' sample data item id.
        self.adapter.add_data_item(self.t1) # adding self.t1 in adapter
```

```

# Virtual sensor (random humidity value)
self.h1 = Sample("h1") # self.h1 takes 'h1' sample data item id.
self.adapter.add_data_item(self.h1) # adding self.h1 in adapter
## Add more samples below, if needed.

# MTConnect adapter availability
self.avail = Event("avail")
self.adapter.add_data_item(self.avail)

# Start MTConnect
self.adapter.start()
self.adapter.begin_gather()
self.avail.set_value("AVAILABLE")
self.adapter.complete_gather()
self.adapter_stream()

def read_temp_raw(self):
    os.system("modprobe w1-gpio")
    os.system("modprobe w1-therm")
    base_dir = "/sys/bus/w1/devices/"
    device_folder = glob.glob(base_dir + "28*")[0]
    device_file = device_folder + "/w1_slave"
    f = open(device_file, "r")
    lines = f.readlines()
    f.close()
    return lines

def read_temp(self):
    lines = self.read_temp_raw()
    while lines[0].strip()[-3:] != "YES":
        time.sleep(0.2)
        lines = self.read_temp_raw()
    equals_pos = lines[1].find("t=")
    if equals_pos != -1:
        temp_string = lines[1][equals_pos + 2 :]
        temp_c = float(temp_string) / 1000.0
        return temp_c

def adapter_stream(self):
    while True:
        try:
            # Do something here
            # To halt this loop, short-cut is CTRL+C
            t1 = self.read_temp() # temperature
            h1 = random.uniform(
                50, 70
            ) # Humidity (random input 50% - 70%)

            now = datetime.datetime.now() # get current data time

            self.adapter.begin_gather() # start to collection
            self.t1.set_value(
                str(t1)
            ) # set SAMPLE value of h1 (temperature) data item
            self.h1.set_value(
                str(h1)
            ) # set SAMPLE value of h1 (humidity) data item
            self.adapter.complete_gather() # end of collection

            print(
                "{} MTConnect data collection completed ... ".format(now)
            ) # Printing out completed MTConnect collection
            print(
                "DS18B20: Temperature={}°C\n".format(t1)
            ) # Printing out DS18B20 measured values
            print("Virtual: Humidity={}RH%\n".format(h1))

            time.sleep(2) # wait for 2 seconds = sampling period

```

```

        except KeyboardInterrupt: # To stop MTConnect adapter, Ctrl + c
            print("Stopping MTConnect...")
            self.adapter.stop() # Stop adapter thread
            sys.exit() # Terminate Python

## ===== MAIN =====
if __name__ == "__main__":
    # start MTConnect Adapter
    MTConnectAdapter("127.0.0.1", 7878) # Args: host ip, port number

#

```

```

In [ ]: # Lab5_adapter2-2.py

# Adapter for ADXL345 sensor
# Sampling period = 1 sec

# Place your code here:
# MTConnect adapter sample for ME597 Lab5
# This code is for accelerometer sensor (measure acceleration)

import sys
import time
import datetime
from data_item import Event, Sample # Load data_item package
from mtconnect_adapter import Adapter # Load mtconnect_adapter package

import board
import busio
import adafruit_adxl34x

class MTConnectAdapter(object): # MTConnect adapter object

    def __init__(self, host, port): # init of MTconnectAdapter class
        # MTConnect adapter connection info
        self.host = host # host arg of adapter
        self.port = port # port arg of adapter
        self.adapter = Adapter((host, port))

        # For samples
        self.a1 = Sample("a1") # self.a1 takes 'a2' sample data item id.
        self.adapter.add_data_item(self.a1) # adding self.a2 in adapter
        self.a2 = Sample("a2") # self.a1 takes 'a1' sample data item id.
        self.adapter.add_data_item(self.a2) # adding self.a3 in adapter
        self.a3 = Sample("a3") # self.a1 takes 'a3' sample data item id.
        self.adapter.add_data_item(self.a3) # adding self.a3 in adapter

        # For events
        ## Add more events below, if needed.

        # MTConnect adapter availability
        self.avail = Event("avail")
        self.adapter.add_data_item(self.avail)

        # Start MTConnect
        self.adapter.start()
        self.adapter.begin_gather()
        self.avail.set_value("AVAILABLE")
        self.adapter.complete_gather()
        self.adapter_stream()

    def adapter_stream(self):
        while True:
            try:
                # Do something here
                # To halt this loop, short-cut is CTRL+C
                a = acc.acceleration # get acceleration
                a1 = a[0] # get X-axis acceleration

```

```

a2 = a[1] # get Y-axis acceleration
a3 = a[2] # get Z-axis acceleration

now = datetime.datetime.now() # get current data time

self.adapter.begin_gather() # start to collection

self.a1.set_value(
    str(a1)
) # set SAMPLE value of a1 (X-axis acceleration) data item
self.a2.set_value(
    str(a2)
) # set SAMPLE value of a2 (Y-axis acceleration) data item
self.a3.set_value(
    str(a3)
) # set SAMPLE value of a3 (Z-axis acceleration) data item

self.adapter.complete_gather() # end of collection

print(
    "{} MTConnect data collection completed ... ".format(now)
) # Printing out completed MTConnect collection
print(
    "ADXL345: Xacc={}, Yacc={}, Zacc={} mm/s^2\n".format(
        a1, a2, a3
    )
) # Printing out ADXL345 measured values

time.sleep(1) # wait for 1 second = Sampling period

except KeyboardInterrupt: # To stop MTConnect adapter, Ctrl + c
    print("Stopping MTConnect...")
    self.adapter.stop() # Stop adapter thread
    sys.exit() # Terminate Python

## ===== MAIN =====
if __name__ == "__main__":
    # i2c variable defines I2C interfaces and GPIO pins using busio and board modules
    i2c = busio.I2C(board.SCL, board.SDA)

    # acc object is instantiation using i2c of Adafruit ADXL34X Library
    acc = adafruit_adxl34x.ADXL345(i2c)

    # start MTConnect Adapter
    MTConnectAdapter("127.0.0.1", 7879) # Args: host ip, port number

#

```

```

In [ ]: # Lab5_adapter2-3.py

# Adapter for power meter
# Sampling period = 0.5 sec

# Place your code here:
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

# MTConnet adapter sample for ME597 Lab5
# This code is for power meter

import sys
import time
import datetime
from data_item import Event, Sample
from mtconnect_adapter import Adapter

from pymodbus.client.sync import ModbusTcpClient
from pymodbus.constants import Endian
from pymodbus.payload import BinaryPayloadDecoder

```

```

# function for power meter
def readReg(client, address, length=2, unit=1):
    rr = client.read_holding_registers(
        address=address, count=length, unit=unit
    )
    if rr is None or rr.isError():
        raise RuntimeError(f"Modbus read failed at address {address}: {rr}")

    decoder = BinaryPayloadDecoder.fromRegisters(
        rr.registers, byteorder=Endian.Big, wordorder=Endian.Big
    )
    return decoder.decode_32bit_float()

class MTConnectAdapter(object):

    def __init__(self, host, port): # init of MTconnectAdapter class
        # MTConnect adapter connection info
        self.host = host
        self.port = port
        self.adapter = Adapter((host, port))

        # For samples
        self.p1 = Sample("p1") # power (W)
        self.adapter.add_data_item(self.p1)

        # For events
        self.ps = Event("ps") # power state
        self.adapter.add_data_item(self.ps)

        # MTConnect adapter availability
        self.avail = Event("avail")
        self.adapter.add_data_item(self.avail)

        # Power meter Modbus TCP info
        self.pm_ip = "10.165.67.146" # IP address of the power meter
        self.pm_port = 502
        self.pm_unit = 1

        self.power_reg_addr = 1536 # Address??
        self.power_reg_len = 2

        # Threshold for ON/OFF
        self.on_threshold_w = 5.0

        # Start MTConnect
        self.adapter.start()
        self.adapter.begin_gather()
        self.avail.set_value("AVAILABLE")
        self.adapter.complete_gather()

        self.adapter_stream()

    def adapter_stream(self):
        while True:
            try:
                c = ModbusTcpClient(self.pm_ip, port=self.pm_port, timeout=2)
                if not c.connect():
                    c.close()
                    raise RuntimeError(
                        f"Failed to connect ModbusTcpClient to {self.pm_ip}:{self.pm_port}"
                    )

                # Read power (W)
                p1 = readReg(
                    c, self.power_reg_addr, self.power_reg_len, self.pm_unit
                )

```

```

# Determine power state
ps = "ON" if p1 > 0 else "OFF"

now = datetime.datetime.now()

self.adapter.begin_gather()
self.p1.set_value(f"{p1:.3f}")
self.ps.set_value(ps)
self.adapter.complete_gather()

print(
    "{} MTConnect data collection completed ... ".format(now)
)
print(
    "Power meter: Machine is now {}, {:.3f} W\n".format(ps, p1)
)

c.close()
time.sleep(0.5)

except KeyboardInterrupt:
    print("Stopping MTConnect...")
    self.adapter.stop()
    sys.exit()

except Exception as e:
    print("Error:", e)
    try:
        c.close()
    except Exception:
        pass
    time.sleep(1)

## ===== MAIN =====
if __name__ == "__main__":
    MTConnectAdapter("127.0.0.1", 7880)

#

```

Task 2.3

1. On your laptop, run the MTConnect adapter for the power meter.
 - Please keep running MTConnect adapters for DS18B20 and ADXL345 on Raspberry Pi.
 - Hint: _ You need to change 'agent.cfg' file. _ You need to run the adapter Python code for power meter on laptop. * If it does not work, please check Windows firewall.
 2. Capture a web browser as Figure 9 after requesting 'current' from the agent and attach it to the report below.
-

MTConnect Device S: X
MTConnect Device S: X
view-source:192.168.50.190:5000/current
rename the current t: X
Or operator xpath h: X
How to combine AN: X

Not secure 192.168.50.190:5000/current
Summarize

Homepage - Purd...
Purdue Portal
John Blair Hooded...
Cash & stock | Sto...
ASUS E-Service
OneCampus | Mos...
Your Courses | Gra...
Other favorites

- creationTime: 2026-02-24T06:38:08Z
- sender: robertclaud
- instanceId: 1771915052
- version: 1.8.0.3
- bufferSize: 131072
- nextSequence: 100
- firstSequence: 1
- lastSequence: 99

Device: sensor; UUID: 000

Device : sensor

Samples

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T06:38:02.415037Z	Acceleration		Xacc	a1	84	-0.3530394
2026-02-24T06:38:07.42317Z	Acceleration		Yacc	a2	95	-0.588399
2026-02-24T06:38:08.424696Z	Acceleration		Zacc	a3	99	9.9243298
2026-02-24T06:38:06.927815Z	HumidityRelative	ACTUAL	humd	h1	94	51.5298859088531
2026-02-24T06:38:07.70883Z	Wattage	ACTUAL	power	p1	97	59.99
2026-02-24T06:37:32.216998Z	Temperature	ACTUAL	temp	t1	11	17.187

Events

Timestamp	Type	Sub Type	Name	Id	Sequence	Value
2026-02-24T06:37:32.216998Z	Availability			avail	13	AVAILABLE
2026-02-24T06:38:07.70883Z	PowerState	LINE	powerstate	ps	98	ON
2026-02-24T06:37:32.163676Z	AssetChanged			s1_asset_chg	2	UNAVAILABLE
2026-02-24T06:37:32.163049Z	AssetRemoved			s1_asset_rem	1	UNAVAILABLE

3. In the text bock below, place changed parts in 'agent.cfg'.

```

Adapter1 {
# This is for DS18B20 & Virtual Sensor
Host = 127.0.0.1
Port = 7878
}
Adapter2 {
# This is for ADXL345 sensor
Host = 127.0.0.1
Port = 7879
}
Adapter3 {
# This is for power meter
Host = 192.168.50.118
Port = 7880
}

```

4. Discuss what changes were made.

In the file lab5_adapter2-1.py, I needed to add the virtual humidity sensor. I completed the file by adding the Sample('h1') sample data id to the initialization function. I then uncommented the line under the adapter_stream function and set the value of h1 to a random number between 50-70. Finally, I ensured that the port used for the MTConnectAdapter was 7878

In the file lab5_adapter2-2.py, I needed to add the acceleration in the z-direction. I did this by adding a self.a3 = Sample('a3') sample data id to the initialization function. I also needed to add a1 to the adapter stream. To do this, I uncommented the line and set its value to a[0], as all three directions of acceleration are retrieved in a tuple with 3

elements. I also changed the print statement to include Xacc. Finally, I ensured that the port used for the MTConnectAdapter was 7879

In the file lab5_adapter2-3.py, I needed to change some of the connection parameters for the power meter. First, I changed the IP address to 10.165.67.146 since I am an online student. I then used the address 1536 from lab5_adapter1.py to connect correctly. I then filled in the values for p1 and ps as follows:

```
p1 = readReg(c, self.power_reg_addr, self.power_reg_len, self.pm_unit)
ps = "ON" if p1 > 0 else "OFF"
```

Also from lab5_adapter1.py. Finally, I ensured that the port used for the MTConnectAdapter was 7880. Since I was running this file locally on my laptop, I also needed to change the host to 0.0.0.0 so that it would be exposed on the network.

In the agent configuration file, I added two additional adapters with comments and ensured that they were looking at the correct ports, 7878, 7879, and 7880 respectively.

Please keep your MTConnect stream working to perform the following Part (5.3).

Please continue to [Lab 5.3 here](#).